



DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: PROJECT EVALUATION (24CS2235F)

A.Y. 2026 - 2027

TITLE OF THE PROJECT	CodeLens — Code Understanding & Impact Analysis	
S. No.	University ID	Name
1	2420030209	Bhanu Prakash
2	2420030443	Krishita Myneni
3	2420030002	Laasya Priya
4	2420030067	Lakshman
Name of the Guide		Dr. P. Krishna Kishore

Abstract

When we work with code written by someone else, it often takes a long time to figure out what the program is actually doing. It gets harder when that code has to be changed. Tools like git diff show which lines were edited, but they don't tell us what else in the program depends on those lines or whether the program will now behave differently. In this project we are building **CodeLens**, a tool that uses compiler techniques to explain how a C program works and to show what effect a change will have on it. We are not using any AI or machine learning. Every result comes from analysing the structure of the program itself. The user pastes or uploads a C program (we support a subset of C) in a web interface. CodeLens first runs it through a compiler front end that we are implementing ourselves. The lexer breaks the code into tokens, the parser checks it against the grammar and builds an Abstract Syntax Tree (AST), and a symbol table records each variable with its type, scope and the lines where it is used. The AST is then converted into Three Address Code (TAC), which is split into basic blocks and connected into a Control Flow Graph (CFG). On top of the CFG we run data-flow analysis, mainly reaching definitions and def-use chains, so we know where each value comes from and where it is used. Using this information, it gives a plain-language explanation of the program: what a function does, how data moves from input to output, which conditions decide the path taken and which paths are possible. Users who want to go deeper can open separate tabs to see the AST, symbol table, TAC, basic blocks, CFG and data-flow results. The second part of the project is change impact analysis. The user gives an old and a new version of the same program. CodeLens builds the AST and CFG for both, compares them to find what changed, and follows that change through the control and data dependencies to find the variables, conditions, blocks and return statements that are affected. For example, if **if (y > 10)** is changed to **if (y >= 10)**, a normal diff only shows that one operator changed. CodeLens points out that the input y = 10 now takes the other branch, so the function returns 15 instead of 5, and shows the chain of reasons behind it. All of this is summarised in a change impact report. Overall, the project covers most of the topics from our Compiler Design course, including lexical analysis, finite automata, context-free grammars, ASTs, symbol tables, intermediate code generation, basic blocks, control flow graphs and data-flow analysis, and applies them to a problem programmers run into in everyday work.

Signature of the Guide

HOD